

① What is JavaScript ? Explain...

JavaScript is a high-level, interpreted programming language mainly used to make web pages interactive and dynamic.

It can be used to :

- Change HTML content
- Modify CSS styles
- Handle user events
- Validate forms
- Create animations
- Communicate with servers
- Build web applications

Example :

```
let name = "Sagar";
console.log("Hello " + name);
```

Output :

Hello Sagar

JavaScript works mainly with HTML and CSS :

HTML	→ Structure of the webpage
CSS	→ Design and appearance
JavaScript	→ Behaviour and interactivity

Example :

```
<button onclick="alert('Hello!')">
  Click Me
</button>
```

When the user clicks the button, JavaScript displays an alert message.

Why is JavaScript important ?

JavaScript is important because it allows developers to create interactive and dynamic websites instead of only static web pages.

JavaScript can run in :

-  Web browsers (Chrome, Firefox, Edge etc.)
-  Servers using Node.js
-  Mobile applications (using frameworks)
-  Desktop applications (using Electron)

In simple words :

JavaScript is used to add logic, behaviour, and interactivity to web pages.

② Features of JavaScript

JavaScript has many features that make it popular for web development. Some important features are :

1. **Lightweight** : JavaScript is a lightweight programming language and easy to learn and use.
2. **Interpreted** : It is an interpreted language, so it runs directly in the browser without compilation.
3. **Dynamic Typing** : Variable types are determined at runtime.
4. **Object - Based** : JavaScript is object-based, which means it uses objects to store data and functions.
5. **Event - Driven** : JavaScript can respond to user events like clicks, keypress, mouse actions etc.
6. **Platform Independent** : It can run on any operating system that has a web browser.
7. **Versatile** : It can be used for front-end, back-end (Node.js), mobile apps and desktop applications.
8. **Large Community** : JavaScript has a large developer community and many libraries and frameworks.

③ JavaScript vs Java

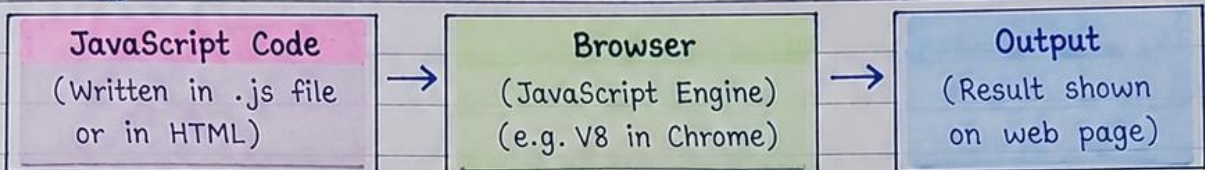
JavaScript and Java are different programming languages. Both have different features and use cases.

Feature	JavaScript	Java
Type	Scripting language	Programming language
Execution	Interpreted	Compiled
Platform	Runs in web browsers	Runs on JVM (Java Virtual Machine)
Typing	Dynamically typed	Statically typed
Use	Mainly for web development	Used for desktop, web, mobile and enterprise apps
Syntax	Simple and flexible	More strict and complex
File extension	.js	.java
Created by	Brendan Eich (Netscape)	James Gosling (Sun Microsystems)

Conclusion : JavaScript is mainly used for making web pages interactive, while Java is a general-purpose programming language.

④ How JavaScript Works ?

JavaScript code is written in a text editor and included in an HTML file. When the web page is loaded in a browser, the browser's JavaScript engine reads and executes the code line by line.



⑤ Ways to Add JavaScript to HTML

There are 3 ways to add JavaScript code to an HTML page :

1. **Internal JavaScript** (Inside `<script>` tag)
2. **External JavaScript** (Using a separate .js file)
3. **Inline JavaScript** (Inside an HTML element)

1. Internal JavaScript :

JavaScript code is written inside the `<script>` tag in the same HTML file.

Example :

```

<!DOCTYPE html>
<html>
<head>
  <script>
    console.log("Hello JavaScript");
  </script>
</head>
</html>
  
```

2. External JavaScript :

JavaScript code is written in a separate .js file and linked using the `<script>` tag.

Example :

```

<script src="script.js"></script>
  
```

3. Inline JavaScript :

JavaScript code is written inside an HTML element using an event.

Example :

```

<button onclick="alert('Hello!')">
  Click Me
</button>
  
```

⑥ JavaScript Syntax

JavaScript syntax is the set of rules for writing JavaScript code. It is similar to C, C++ and Java.

Example :

```

let name = "Sagar";
console.log(name);
  
```

Rules :

- Statements end with a semicolon (;).
- JavaScript is case-sensitive
- Use meaningful variable names
- Spaces and line breaks are ignored

7 Variables in JavaScript

A variable is a container used to store data in memory. We can use the stored data later in our program.

Example :

```
let name = "Sagar";
let age = 19;
console.log(name);    // Sagar
console.log(age);     // 19
```

Output :

```
Sagar
19
```

8 Types of Variable Declaration

In JavaScript, we can declare variables using three keywords:

1. **var** (Old way, function scoped)
2. **let** (Block scoped, can be updated)
3. **const** (Block scoped, cannot be updated)

1. var (Function Scoped)

- Introduced in ES5 (old version).
- Can be re-declared and updated.
- Function scoped (not block scoped).

Example :

```
var x = 10;
var x = 20;    // re-declared
x = 30;        // updated
console.log(x); // 30
```

2. let (Block Scoped)

- Introduced in ES6.
- Cannot be re-declared in the same scope.
- Can be updated.
- Block scoped { }.

Example :

```
let y = 10;
// let y = 20;    // Error (re-declaration)
y = 20;           // allowed
console.log(x);   // 20
```

3. const (Block Scoped)

- Introduced in ES6.
- Cannot be re-declared.
- Cannot be updated.
- Must be initialized at the time of declaration.
- Block scoped { }.

Example :

```
const pi = 3.14;
// pi = 3.15;    // Error (cannot update)
// const pi = 22/7; // Error (re-declaration)
console.log(pi); // 3.14
```

Comparison of var, let and const

Feature	var	let	const
Introduced in	ES5	ES6	ES6
Scope	Function scoped	Block scoped	Block scoped
Re-declaration	Allowed	Not allowed	Not allowed
Updating value	Allowed	Allowed	Not allowed
Initialization	Optional	Optional	Mandatory
Best use	Avoid using	General use	Fixed values (constants)

9 Data Types in JavaScript

A data type defines what kind of value a variable can store.

JavaScript has two types of data types:

1. Primitive (Basic) Data Types
2. Non-Primitive (Reference) Data Types

1. Primitive Data Types :

These are basic data types and store single values.

There are 7 primitive data types in JavaScript:

1. String
2. Number
3. Boolean
4. Undefined
5. Null
6. Symbol
7. BigInt

Example :

```
let name = "Sagar";    // String
let age = 19;          // Number
let isStudent = true;  // Boolean
let x;                 // Undefined
let y = null;          // Null
let id = Symbol("id"); // Symbol
let big = 123456789n;  // BigInt
```

2. Non-Primitive Data Types :

These are reference types and can store multiple values.

There are 3 non-primitive data types:

1. Object
2. Array
3. Function

Example :

```
let person = { name: "Sagar", age: 19 }; // Object
let colors = ["red", "green", "blue"];   // Array
let show = function() {                  // Function
  console.log("Hello");
};
```

10 Type Checking in JavaScript

We can check the data type of a variable using the `typeof` operator.

Example :

```
let name = "Sagar";
console.log(typeof name);

let age = 19;
console.log(typeof age);

let isStudent = true;
console.log(typeof isStudent);
```

Output :

```
string
number
boolean
```

Note :

- `typeof null` returns "object"
- `typeof array` returns "object"
- `typeof function` returns "function"

11 Operators in JavaScript

Operators are used to perform operations on values and variables.

Type of Operator	Operators	Example	Description
1. Arithmetic	+ - * / % **	let a = 5 + 3;	Perform mathematical operations
2. Assignment	= += -= *= /= %=	let x = 10;	Assign values
3. Comparison	== === != !== > < >= <=	5 == "5"	Compare two values
4. Logical	&& !	(a > 5) && (b < 10)	Combine conditions
5. Unary	+ - ++ -- !	let x = -5;	Operate on single value
6. Ternary	? :	let r = (a > b) ? a : b;	Short hand if-else

12

Type Conversion and Type Coercion in JavaScript

JavaScript can convert values from one data type to another. This is called type conversion. JavaScript also automatically converts types in some cases, which is called type coercion.

1. Type Conversion (Manual Conversion)

We can manually convert a value from one type to another using built-in functions.

Conversion	Method	Example	Result
String to Number	Number()	Number("10")	10
Number to String	String()	String(20)	"20"
String to Integer	parseInt()	parseInt("25")	25
String to Float	parseFloat()	parseFloat("3.14")	3.14
Any to Boolean	Boolean()	Boolean(1)	true

Example :

```
let num = "100";
let n = Number(num);
console.log(n); // 100

let str = 50;
let s = String(str);
console.log(s); // "50"

let val = "25.7";
console.log(parseInt(val)); // 25
console.log(parseFloat(val)); // 25.7
```

2. Type Coercion (Automatic Conversion)

JavaScript automatically converts types when performing operations.

- "5" + 2 → "52" (String + Number = String)
- "5" - 2 → 3 (String converted to Number)
- true + 1 → 2 (true becomes 1)
- false + 1 → 1 (false becomes 0)

Example :

```
console.log("5" + 2); // "52"
console.log("5" - 2); // 3
console.log(true + 1); // 2
console.log(false + 1); // 1
```

Note :

- Use type conversion when you want to explicitly change the type.
- Be careful with type coercion as it can lead to unexpected results.

13

Conditional Statements in JavaScript

Conditional statements are used to execute a block of code based on a condition. JavaScript provides the following conditional statements:

1. if statement
2. if...else statement
3. if...else if...else statement
4. switch statement

1. if statement :

It executes a block of code if the condition is true.

```
let age = 18;
if (age >= 18) {
  console.log("You are an adult");
}
```

2. if...else statement :

It executes one block if the condition is true, otherwise another block.

```
let age = 16;
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

3. if...else if...else statement :

It checks multiple conditions.

```
let marks = 75;
if (marks >= 90) {
  console.log("Grade A");
} else if (marks >= 60) {
  console.log("Grade B");
} else {
  console.log("Grade C");
}
```

4. switch statement :

It is used to execute one block of code from multiple options.

```
let day = 3;
switch(day) {
  case 1: console.log("Monday"); break;
  case 2: console.log("Tuesday"); break;
}
```

Note :

- if → used for single condition.
- if...else → used for two conditions.
- if...else if...else → used for multiple conditions.
- switch → used when there are multiple fixed

14

Loops in JavaScript

Loops are used to execute a block of code repeatedly until a specific condition is satisfied. JavaScript provides different types of loops.

Types of Loops in JavaScript :

1. for loop
2. while loop
3. do...while loop
4. for...in loop (used with objects)
5. for...of loop (used with arrays and iterable objects)

1. for loop :

It is used to execute a block of code a fixed number of times.

Syntax :

```
for (initialization; condition; update) {
    // code to be executed
}
```

Example :

```
for (let i = 1; i <= 5; i++) {
    console.log("Hello " + i);
}
```

Output :

```
Hello 1
Hello 2
Hello 3
Hello 4
Hello 5
```

2. while loop :

It executes a block of code as long as the condition is true.

Syntax :

```
while (condition) {
    // code to be executed
}
```

Example :

```
let i = 1;
while (i <= 5) {
    console.log(i);
    i++;
}
```

Output :

```
1
2
3
4
5
```

3. do...while loop :

It is similar to while loop, but it executes the block of code at least once, even if the condition is false.

Syntax :

```
do {
    // code to be executed
} while (condition);
```

Example :

```
let i = 1;
do {
    console.log(i);
    i++;
} while (i <= 5);
```

Output :

```
1
2
3
4
5
```

4. for...in loop :

It is used to iterate over the properties of an object.

Example :

```
let student = {
    name: "Sagar",
    age: 19,
    course: "MCA"
};
for (let key in student) {
    console.log(key + " : " + student[key]);
}
```

Output :

```
name : Sagar
age : 19
course : MCA
```

5. for...of loop :

It is used to iterate over the values of an iterable object like array.

Example :

```
let colors = ["red", "green", "blue"];
for (let color of colors) {
```

Output :

```
red
green
```

Note :

- Use for loop when the number of iterations is known.
- Use while loop when the number of iterations is unknown.
- Use do...while when you want to execute at least once.
- Use for...in with objects.
- Use for...of with arrays or iterable objects.

15

Functions in JavaScript

A function is a block of code that performs a specific task. Functions help to make the code reusable, organized and easy to maintain.

1. Function Declaration :

It is the normal way to define a function using the function keyword.

Syntax :

```
function functionName(parameters) {
    // code to be executed
    return value; // optional
}
```

Example :

```
function greet() {
    console.log("Hello Sagar");
}
greet(); // function call
```

Output :

Hello Sagar

2. Function with Parameters :

We can pass values to a function using parameters.

Example :

```
function add(a, b) {
    return a + b;
}
let result = add(5, 3);
console.log(result);
```

Output :

8

Note :

- Parameters are values passed to a function.
- A function can have multiple parameters.
- We can also return a value from a function using return.

3. Function with Default Parameter :

We can provide a default value to a parameter.

Example :

```
function greet(name = "User") {
    console.log("Hello " + name);
}
greet(); // Hello User
greet("Sagar"); // Hello Sagar
```

Output :

Hello User
Hello Sagar

Note :

- If no value is passed, the default value is used.
- Default parameters were introduced in ES6.

16

Arrow Functions in JavaScript

Arrow functions are a shorter way to write functions. They were introduced in ES6 and use the `=>` (arrow) syntax.

Syntax :

```
(parameters) => {
    // code to be executed
    return value;
}
```

Example :

```
const add = (a, b) => {
    return a + b;
};
console.log(add(5, 3));
```

Output :

8

Different Forms of Arrow Function :

Type	Syntax	Example	Output
1. Single parameter	<code>(a) => a * 2</code>	<code>let x = a => a * 2;</code> <code>console.log(x(5));</code>	10
2. Multiple parameters	<code>(a, b) => a + b</code>	<code>let x = (a, b) => a + b;</code> <code>console.log(x(5, 3));</code>	8

Note :

- Arrow functions are shorter than normal functions.
- They do not have their own this (important in objects).
- Arrow functions are best suited for short functions.

17

Arrays in JavaScript

An array is a special variable that can store multiple values in a single variable. Array can store values of different data types like numbers, strings, booleans, objects etc.

1. Array Declaration :

We can declare an array using square brackets [].

Example :

```
let numbers = [10, 20, 30, 40];
let names = ["Sagar", "Rahul", "Amit"];
let mixed = [10, "Sagar", true, 25.5];
```

Output :

```
numbers: [10, 20, 30, 40]
names: ["Sagar", "Rahul", "Amit"]
mixed: [10, "Sagar", true, 25.5]
```

2. Accessing Array Elements :

We can access array elements using index. Index starts from 0.

Example :

```
let colors = ["red", "green", "blue"];
console.log(colors[0]); // red
console.log(colors[1]); // green
console.log(colors[2]); // blue
```

Output :

```
red
green
blue
```

Note :

- Index always starts from 0.
- If we try to access an index that does not exist, it returns undefined.

3. Array Length :

The length property returns the number of elements in an array.

Example :

```
let numbers = [10, 20, 30, 40, 50];
console.log(numbers.length);
```

Output :

```
5
```

4. Modifying Array Elements :

We can change the value of an array element using its index.

Example :

```
let fruits = ["apple", "banana", "mango"];
fruits[1] = "orange";
console.log(fruits);
```

Output :

```
["apple",
 "orange",
 "mango"]
```

5. Adding New Element :

We can add a new element using push(). It adds element at the end.

Example :

```
let nums = [1, 2, 3];
nums.push(4);
console.log(nums);
```

Output :

```
[1, 2, 3, 4]
```

6. Removing Last Element :

We can remove the last element using pop().

Example :

```
let nums = [1, 2, 3, 4];
nums.pop();
console.log(nums);
```

Output :

```
[1, 2, 3]
```

Common Array Methods :

Method	Description
push()	Add element at end
pop()	Remove last element
shift()	Remove first element
unshift()	Add element at start
length	Return number of elements
indexOf()	Return index of element
includes()	Check if element exists (returns true/false)
slice()	Returns a portion of array
splice()	Add/Remove elements from specific index
sort()	Sort elements
reverse()	Reverse the array

18

Array Iteration and Useful Methods in JavaScript

We can iterate over array elements and perform various operations using loops and built-in array methods.

1. Iterating using for loop :

We can use a for loop to iterate over the array elements.

Example :

```
let fruits = ["apple", "banana", "mango"];
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

Output :

```
apple
banana
mango
```

Note :

- We use `fruits.length` to get the number of elements.
- Index starts from 0.

2. Iterating using for...of loop :

The for...of loop directly gives each element of the array.

Example :

```
let fruits = ["apple", "banana", "mango"];
for (let fruit of fruits) {
  console.log(fruit);
}
```

Output :

```
apple
banana
mango
```

Note :

- `for...of` is easy to use.
- It directly gives the element without using index.

3. Iterating using forEach() :

The `forEach()` method executes a function for each element in the array.

Example :

```
let fruits = ["apple", "banana", "mango"];
fruits.forEach(function(fruit, index) {
  console.log(index + " : " + fruit);
});
```

Output :

```
0 : apple
1 : banana
2 : mango
```

Note :

- `forEach()` takes a function with element and index as parameters.
- It does not return a new array.

4. map() Method :

The `map()` method creates a new array by applying a function to each element.

Example :

```
let nums = [1, 2, 3, 4];
let squares = nums.map(function(n) {
  return n * n;
});
console.log(squares);
```

Output :

```
[1, 4, 9, 16]
```

Note :

- `map()` returns a new array.
- It does not modify the original array.

5. filter() Method :

The `filter()` method creates a new array with elements that satisfy a condition.

Example :

```
let nums = [10, 20, 30, 40, 50];
let evenNums = nums.filter(function(n) {
  return n % 2 === 0;
});
console.log(evenNums);
```

Output :

```
[20, 40]
```

Note :

- `filter()` returns a new array with elements that pass the condition.
- It does not change the original array.